

Fəsil 18. Reinforcement Learning

Reinforcement learning (RL) bu gün maşın öyrənməsinin ən həyəcanverici sahələrindən biridir və həmçinin ən qədimlərindən biridir. O, 1950-ci illərdən bəri mövcuddur və illər ərzində bir çox maraqlı tətbiqlər, xüsusən oyunlar (məsələn, *TD-Gammon*, bir Backgammon oynayan proqram) və maşın idarəetməsi sahələrində ortaya çıxmışdır, lakin nadir hallarda başlıq xəbərlərinə çıxmışdır. Lakin 2013-cü ildə DeepMind adlı Britaniya startapından olan tədqiqatçılar demək olar ki, istənilən Atari oyununu sıfırdan öyrənə bilən bir sistem nümayiş etdirdikdə bir inqilab baş verdi. Bu sistem nəticədə əksər oyunlarda insanları üstələdi, yalnız xam pikselləri giriş kimi istifadə edərək və oyunların qaydaları haqqında heç bir əvvəlki bilik olmadan. Bu, bir sıra heyranəmiz uğurların birincisi idi və 2016-cı ilin martında onların sistemi AlphaGo-nun əfsanəvi professional Go oyunçusu Lee Sedol-a, 2017-ci ilin mayında isə dünya çempionu Ke Jie-yə qalib gəlməsi ilə zirvəyə çatdı. Heç bir proqram bu oyunun ustasını məğlub etməyə yaxınlaşmamışdı, dünya çempionu bir yana. Bu gün bütün RL sahəsi yeni ideyalarla qaynayır və geniş tətbiq sahələrinə malikdir.

Bəs DeepMind (2014-cü ildə Google tərəfindən 500 milyon dollardan çox qiymətə satın alındı) bütün bunlara necə nail oldu? Geriyə baxanda olduqca sadə görünür: onlar deep learning-in gücünü reinforcement learning sahəsinə tətbiq etdilər və bu, onların ən cəsarətli arzularından da kənarda işlədi. Bu fəsildə mən əvvəlcə reinforcement learning-in nə olduğunu və nədə yaxşı olduğunu izah edəcəyəm, sonra dərin reinforcement learning-də ən mühüm iki texnikanı təqdim edəcəyəm: policy gradients və deep Q-networks, o cümlədən Markov decision processes-in müzakirəsi. Başlayaq!

Mükafatları Optimallaşdırmağı Öyrənmək

Reinforcement learning-də bir proqram *agent* i *environment* (mühit) daxilində *observations* (müşahidələr) edir və *actions* (hərəkətlər) yerinə yetirir, müqabilində environment-dən *rewards* (mükafatlar) alır. Onun məqsədi zamanla gözlənilən mükafatları maksimuma çıxaracaq şəkildə hərəkət etməyi öyrənməkdir. Əgər bir az antropomorfizmə etiraz etmirsinizsə, müsbət mükafatları zövq, mənfi mükafatları isə ağrı kimi düşünə bilərsiniz (bu halda "reward" termini bir az yanıltıcıdır). Qısacası, agent environment-də hərəkət edir və sınaq və səhv yolu ilə zövqünü maksimuma, ağrısını isə minimuma endirməyi öyrənir.

Bu olduqca geniş bir çərçivədir və müxtəlif tapşırıqlara tətbiq edilə bilər. Budur bir neçə nümunə (bax Şəkil 18-1):

1. Agent, bir robotu idarə edən proqram ola bilər. Bu halda environment real dünyadır, agent environment-i kameralar və toxunma sensorları kimi *sensors* vasitəsilə müşahidə edir və onun hərəkətləri mühərrikləri aktivləşdirmək üçün siqnallar göndərməkdən ibarətdir. O, hədəf yerinə yaxınlaşdıqda müsbət mükafatlar, vaxt itirdikdə və ya səhv istiqamətə getdikdə mənfi mükafatlar almaq üçün proqramlaşdırıla bilər.
2. Agent, *Ms. Pac-Man* oyununu idarə edən proqram ola bilər. Bu halda environment Atari oyununun simulyasiyasıdır, hərəkətlər doqquz mümkün joystick mövqeyidir (yuxarı sol, aşağı, mərkəz və s.), müşahidələr ekran görüntüləridir və mükafatlar sadəcə oyun xallarıdır.
3. Eynilə, agent Go kimi bir stolüstü oyunu oynayan proqram ola bilər. O, yalnız qalib gəldiyi təqdirdə mükafat alır.
4. Agent fiziki (və ya virtual) hərəkət edən bir şeyi idarə etmək məcburiyyətində deyil. Məsələn, o, ağıllı termostat ola bilər, hədəf temperatura yaxın olduqda və enerjiyə qənaət etdikdə müsbət mükafatlar, insanlar temperaturu tənzimləməli olduqda mənfi mükafatlar alır, beləliklə agent insan ehtiyaclarını qabaqcadan görməyi öyrənməlidir.

[Şəkil 18-1] – RL tətbiqlərinin nümunələri: robot, *Ms. Pac-Man*, Go, termostat.

Reinforcement learning-in əsas sxemi Şəkil 18-2-də göstərilmişdir: agent environment-dən müşahidə alır, bir hərəkət edir, mükafat və növbəti müşahidə alır.

[Şəkil 18-2] – RL-nin əsas sxemi: Agent Environment ilə qarşılıqlı əlaqədədir.

OpenAI Gym-ə Giriş

Reinforcement learning-in çətinliklərindən biri, bir agenti öyrətmək üçün əvvəlcə işlək bir environment-ə sahib olmanız lazımdır. Atari oyununu oynamağı öyrənəcək bir agent proqramlaşdırmaq istəyirsinizsə, Atari oyun simulyatoruna ehtiyacınız var. Gəzən bir robot proqramlaşdırmaq istəyirsinizsə, environment real dünyadır və robotunuzu birbaşa həmin environment-də öyrədə bilərsiniz. Lakin bunun məhdudiyyətləri var: əgər robot uçurumdan yığılırsa, sadəcə "Geri al" düyməsini basa bilməzsiniz. Vaxtı da sürətləndirə bilməzsiniz – daha çox hesablama gücü əlavə etmək robotu daha sürətli hərəkət etdirməyəcək – və ümumiyyətlə 1,000 robotu paralel öyrətmək çox baha başa gəlir. Qısacası, real dünyada təlim çətin və yavaşdır, buna görə adətən heç olmasa başlanğıc təlim üçün bir *simulated environment* (simulyasiya edilmiş mühit) lazımdır. Məsələn, 3D fizika simulyasiyası üçün PyBullet və ya MuJoCo kimi kitabxanalardan istifadə edə bilərsiniz.

OpenAI Gym geniş çeşiddə simulyasiya edilmiş environment-lər (Atari oyunları, stolüstü oyunlar, 2D və 3D fiziki simulyasiyalar və s.) təqdim edən bir alət dəstidir. Siz

bunlardan agentləri öyrətmək, onları müqayisə etmək və ya yeni RL alqoritmləri hazırlamaq üçün istifadə edə bilərsiniz.

OpenAI Gym Colab-da əvvəlcədən quraşdırılmışdır, lakin köhnə versiyadır, buna görə onu ən son versiya ilə əvəz etməli olacaqsınız. Onun bəzi asılılıqlarını da quraşdırmalısınız. Əgər Colab əvəzinə öz maşınıızda kodlaşdırırsınızsa və <https://homl.info/install> ünvanındakı quraşdırma təlimatlarına əməl etmisinizsə, bu addımı atlaya bilərsiniz; əks halda, bu əmrləri daxil edin:

```
# Only run these commands on Colab or Kaggle!
```

```
%pip install -q -U gym
```

```
%pip install -q -U gym[classic_control,box2d,atari,accept-rom-license]
```

Birinci `%pip` əmri Gym-i ən son versiyaya yeniləyir. `-q` seçimi *quiet* (sakit) deməkdir: çıxışı daha az şifahi edir. `-U` seçimi *upgrade* (yeniləmə) deməkdir. İkinci `%pip` əmri müxtəlif növ environment-ləri işə salmaq üçün tələb olunan kitabxanaları quraşdırır. Buraya *control theory* – dinamik sistemlərin idarə edilməsi elmi – klassik environment-ləri, məsələn, bir arabanın üzərindəki dirəyi tarazlaşdırmaq daxildir. O, həmçinin Box2D kitabxanasına əsaslanan environment-ləri – oyunlar üçün 2D fizika mühərriki – əhatə edir. Nəhayət, Arcade Learning Environment (ALE) əsaslanan environment-ləri, yəni Atari 2600 oyunları üçün emulyatoru əhatə edir. Bir neçə Atari oyun ROM-u avtomatik olaraq yüklənir və bu kodu işlətməklə siz Atari-nin ROM lisenziyaları ilə razılaşırırsınız.

Bununla, OpenAI Gym-dən istifadə etməyə hazırsınız. Gəlin onu idxal edək və bir environment yaradaq:

```
import gym
```

```
env = gym.make("CartPole-v1", render_mode="rgb_array")
```

Burada biz bir CartPole environment-i yaratdıq. Bu, 2D simulyasiyadır, burada bir araba üzərinə qoyulmuş dirəyi tarazlaşdırmaq üçün sola və ya sağa sürətləndirilə bilər (bax Şəkil 18-4). Bu, klassik bir idarəetmə tapşırığıdır.

İPUCU

`gym.envs.registry` lüğəti bütün mövcud environment-lərin adlarını və spesifikasiyalarını ehtiva edir.

[Şəkil 18-4] – CartPole environment-i: araba və dirək.

`gym.make` funksiyası ilə `environment` yaradarkən `render_mode="rgb_array"` təyin etsək, şəkil NumPy massivi kimi qaytarılacaq:

```
>>> img = env.render()
>>> img.shape # height, width, channels (3 = Red, Green, Blue)
(400, 600, 3)
```

Daha sonra, adətən olduğu kimi, bu şəkli göstərmək üçün Matplotlib-in `imshow()` funksiyasından istifadə edə bilərsiniz.

İndi `environment`-dan hansı hərəkətlərin mümkün olduğunu soruşaq:

```
>>> env.action_space
Discrete(2)
```

`Discrete(2)` o deməkdir ki, mümkün hərəkətlər 0 və 1 tam ədədləridir, bunlar sola və ya sağa sürətləndirməyi təmsil edir. Digər `environment`-lərdə əlavə diskret hərəkətlər və ya digər növ hərəkətlər (məsələn, davamlı) ola bilər. Dirək sağa doğru əyildiyi üçün (`obs[2] > 0`), gəlin arabanı sağa doğru sürətləndirək:

```
>>> action = 1 # accelerate right
>>> obs, reward, done, truncated, info = env.step(action)
>>> obs
array([ 0.02727336,  0.18847767,  0.03625453, -0.26141977], dtype=float32)
>>> reward
1.0
>>> done
False
>>> truncated
False
>>> info
{}
```

`step()` metodu istənilən hərəkəti yerinə yetirir və beş dəyər qaytarır:

obs

Bu, yeni müşahidədir. Araba indi sağa doğru hərəkət edir (`obs[1] > 0`). Dirək hələ də sağa doğru əyilib (`obs[2] > 0`), lakin onun bucaq sürəti indi mənfidir (`obs[3] < 0`), buna görə də növbəti addımdan sonra sola doğru əyiləcək.

reward

Bu environment-də, nə etsəniz, hər addımda 1.0 mükafat alırsınız, buna görə məqsəd epizodu mümkün qədər uzun müddət davam etdirməkdir.

done

Bu dəyər epizod bitdikdə True olacaq. Bu, dirək çox əyildikdə, ya da ekrandan kənara çıxdıqda və ya 200 addımdan sonra baş verəcək (sonuncu halda, siz qalib gəlmisiniz). Bundan sonra, environment yenidən istifadə edilməzdən əvvəl sıfırlanmalıdır.

truncated

Bu dəyər, bir epizod erkən kəsildikdə True olacaq, məsələn, environment wrapper-ı tərəfindən epizod başına maksimum addım sayı tətbiq edildikdə (environment wrapper-ları haqqında daha ətraflı məlumat üçün Gym-in sənədlərinə baxın). Bəzi RL alqoritmləri truncated epizodları normal şəkildə bitən epizodlardan fərqli şəkildə müalicə edir, lakin bu fəsildə biz onları eyni şəkildə müalicə edəcəyik.

info

Bu environment-a xas lüğət əlavə məlumat təqdim edə bilər, `reset()` metodu tərəfindən qaytarılan kimi.

İPUCU

Environment-dən istifadəni bitirdikdən sonra, resursları boşaltmaq üçün onun `close()` metodunu çağırmalısınız.

Gəlin dirək sola əyildikdə sola, sağa əyildikdə isə sağa sürətləndirən sadə bir policy (siyasət) kodlayaq. Bu siyasəti 500 epizod üzərində işlədək və ortalama mükafatı görək:

```
def basic_policy(obs):
    angle = obs[2]
    return 0 if angle < 0 else 1

totals = []
for episode in range(500):
    episode_rewards = 0
    obs, info = env.reset(seed=episode)
    for step in range(200):
        action = basic_policy(obs)
        obs, reward, done, truncated, info = env.step(action)
```

```
episode_rewards += reward
if done or truncated:
    break
```

```
totals.append(episode_rewards)
```

Bu kod özünü izah edir. Nəticəyə baxaq:

```
>>> import numpy as np
>>> np.mean(totals), np.std(totals), min(totals), max(totals)

(41.698, 8.389445512070509, 24.0, 63.0)
```

500 cəhdə belə, bu siyasət heç vaxt dirəyi 63 ardıcıl addımdan çox şaquli saxlaya bilmədi. Pis deyil, amma yaxşı da deyil. Bu fəslin notebook-undakı simulyasiyaya baxsanız, arabanın getdikcə daha güclü sola və sağa salındığını, dirək çox əyilənə qədər davam etdiyini görərsiniz. Gəlin görək bir neyron şəbəkə daha yaxşı bir siyasət tapa bilərmə?

[Şəkil 18-5] – Policy net: müşahidələri qəbul edir, hərəkət ehtimallarını çıxarır.

Bir neyron şəbəkə siyasəti qurmaq üçün Keras-dan istifadə edən kod:

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(5, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid"),
])
```

Biz siyasət şəbəkəsini təyin etmək üçün Sequential modelindən istifadə edirik. Girişlərin sayı müşahidə fəzasının ölçüsüdür – CartPole-da bu 4-dür – və biz yalnız beş gizli vahidə sahibik, çünki bu kifayət qədər sadə bir tapşırıqdır. Nəhayət, biz tək bir ehtimal – sola getmə ehtimalı – çıxarmaq istəyirik, buna görə də sigmoid aktivasiya funksiyasından istifadə edən tək bir çıxış neyronumuz var. İkidən çox mümkün hərəkət olsaydı, hər bir hərəkət üçün bir çıxış neyronu olardı və biz softmax aktivasiya funksiyasından istifadə edərdik.

Yaxşı, indi müşahidələri qəbul edən və hərəkət ehtimallarını çıxaran bir neyron şəbəkə siyasətimiz var. Bəs onu necə öyrədək?

Hərəkətlərin Qiymətləndirilməsi: Kredit Bölgüsü Problemi

Əgər hər addımda ən yaxşı hərəkətin nə olduğunu bilsəydik, təxmin edilən ehtimal paylanması ilə hədəf ehtimal paylanması arasındakı çarpaz entropiyanı minimuma endirərək neyron şəbəkəni adi qaydada öyrədə bilərdik. Bu, adi supervised learning olardı. Lakin reinforcement learning-də agentin aldığı yeganə bələdçilik mükafatlar vasitəsilədir və mükafatlar adətən seyrək və gecikmişdir. Məsələn, əgər agent dirəyi 100 addım tarazlaşdırmağı bacarırsa, onun etdiyi 100 hərəkətdən hansının yaxşı, hansının pis olduğunu necə bilə bilər? Onun bildiyi tək şey, son hərəkətdən sonra dirəyin yıxıldığıdır, lakin şübhəsiz ki, bu son hərəkət tamamilə məsuliyyətli deyil. Buna *credit assignment problem* (kredit bölgüsü problemi) deyilir: agent mükafat aldıqda, hansı hərəkətlərin ona kredit (və ya qınama) qazandırdığını bilmək çətindir. Yaxşı davrandıqdan saatlarla sonra mükafatlandırılan bir iti düşünün; nə üçün mükafatlandırıldığını başa düşəcəkmiz?

Bu problemi həll etmək üçün ümumi bir strategiya, bir hərəkəti ondan sonra gələn bütün mükafatların cəminə əsasən qiymətləndirməkdir, adətən hər addımda bir *discount factor* (diskont amili), γ (qamma) tətbiq etməklə. Diskontlaşdırılmış mükafatların bu cəminə hərəkətin *return* -u (qaytarımı) deyilir. Şəkil 18-6-dakı nümunəni nəzərdən keçirək. Əgər bir agent ardıcıl olaraq üç dəfə sağa getməyə qərar verirsə və ilk addımdan sonra +10, ikinci addımdan sonra 0 və nəhayət üçüncü addımdan sonra -50 mükafat alırsa, 2 diskont amili $\gamma = 0.8$ istifadə etdiyimizi fərz etsək, ilk hərəkətin return-u $10 + \gamma \times 0 + \gamma \times (-50) = -22$ olacaq. Əgər diskont amili 0-a yaxındırsa, gələcək mükafatlar dərhal mükafatlarla müqayisədə çox sayılmayacaq. Əksinə, əgər diskont amili 1-ə yaxındırsa, uzaq gələcəkdəki mükafatlar demək olar ki, dərhal mükafatlar qədər sayılacaq. Tipik diskont amilləri 0.9 ilə 0.99 arasında dəyişir. 0.95 diskont amili ilə, 13 addım sonrakı mükafatlar dərhal mükafatların təxminən yarısı qədər sayılır (çünki $0.95^{13} \approx 0.5$), 0.99 diskont amili ilə isə 69 addım sonrakı mükafatlar dərhal mükafatların yarısı qədər sayılır. CartPole environment-də hərəkətlər kifayət qədər qısamüddətli təsirlərə malikdir, buna görə 0.95 diskont amili seçmək ağlabatan görünür.

[Şəkil 18-6] – Diskontlaşdırılmış gələcək mükafatların hesablanması.

Policy Gradients

Daha əvvəl müzakirə edildiyi kimi, PG alqoritmləri daha yüksək mükafatlara doğru qradientləri izləməklə bir siyasətin parametrlərini optimallaşdırır. PG alqoritmlərinin məşhur bir sinfi, *REINFORCE alqoritmləri* adlanır, 1992-ci ildə Ronald Williams tərəfindən təqdim edilmişdir. Budur onun ümumi bir variantı:

1. Əvvəlcə, neyron şəbəkə siyasətinin oyunu bir neçə dəfə oynamasına icazə verin və hər addımda, seçilmiş hərəkətin daha çox ehtimal olmasını təmin edəcək qradiyentləri hesablayın – lakin bu qradiyentləri hələ tətbiq etməyin.
2. Bir neçə epizod işlətdikdən sonra, əvvəlki bölmədə təsvir edilən metoddan istifadə edərək hər bir hərəkətin advantage (üstünlük) dəyərini hesablayın.
3. Əgər bir hərəkətin advantage dəyəri müsbətdirsə, bu, hərəkətin yaxın ki, yaxşı olduğunu göstərir və əvvəllər hesablanmış qradiyentləri tətbiq edərək həmin hərəkətin gələcəkdə daha çox seçilmə ehtimalını artırmaq istəyirsiniz. Lakin, əgər hərəkətin advantage dəyəri mənfidirsə, bu, hərəkətin yaxın ki, pis olduğunu göstərir və əks qradiyentləri tətbiq edərək bu hərəkətin gələcəkdə bir az *daha az* seçilməsini istəyirsiniz. Həll, hər bir qradiyent vektorunu müvafiq hərəkətin advantage dəyərinə vurmaqdır.
4. Nəhayət, bütün nəticədə yaranan qradiyent vektorlarının ortalamasını hesablayın və gradient descent addımını yerinə yetirmək üçün istifadə edin.

Gəlin Keras-dan istifadə edərək bu alqoritmi tətbiq edək. Daha əvvəl qurduğumuz neyron şəbəkə siyasətini dirəyi arabanın üzərində tarazlaşdırmağı öyrənmək üçün öyrədəcəyik. Əvvəlcə, bir addım oynayacaq bir funksiya ehtiyacımız var. Hələlik, onun etdiyi hər hansı hərəkətin düzgün olduğunu fərz edəcəyik ki, loss və onun qradiyentlərini hesablaya bilək. Bu qradiyentlər bir müddət saxlanılacaq və daha sonra hərəkətin nə qədər yaxşı və ya pis olduğundan asılı olaraq dəyişdiriləcək:

```
def play_one_step(env, obs, model, loss_fn):
    with tf.GradientTape() as tape:
        left_proba = model(obs[np.newaxis])
        action = (tf.random.uniform([1, 1]) > left_proba)
        y_target = tf.constant([[1.]]) - tf.cast(action, tf.float32)
        loss = tf.reduce_mean(loss_fn(y_target, left_proba))
        grads = tape.gradient(loss, model.trainable_variables)
        obs, reward, done, truncated, info = env.step(int(action))

    return obs, reward, done, truncated, grads
```

Bu funksiyanı nəzərdən keçirək:

- GradientTape bloku daxilində (bax Fəsil 12), modelə tək bir müşahidə verərək çağırmaqla başlayırıq. Müşahidəni elə formalaşdırırıq ki, o, tək bir instansdan ibarət batch-ə çevrilsin, çünki model batch gözləyir. Bu, sola getmə ehtimalını çıxarır.
- Sonra, 0 ilə 1 arasında təsadüfi bir float nümunəsi götürürük və onun `left_proba`-dan böyük olub-olmadığını yoxlayırıq. Hərəkət `left_proba` ehtimalı ilə False, ya da `1 - left_proba` ehtimalı ilə True olacaq. Bu Boolean-u

tam ədədə çevirdikdən sonra, hərəkət müvafiq ehtimallarla 0 (sol) və ya 1 (sağ) olacaq.

- İndi sola getmənin hədəf ehtimalını təyin edirik: bu, 1 mənfi hərəkətdir (floata çevrilmiş). Əgər hərəkət 0 (sol) olarsa, sola getmənin hədəf ehtimalı 1 olacaq. Əgər hərəkət 1 (sağ) olarsa, hədəf ehtimalı 0 olacaq.
- Sonra verilmiş loss funksiyasından istifadə edərək loss-u hesablayırıq və tape-dən istifadə edərək loss-un modelin trainable dəyişənlərinə görə qradientini hesablayırıq. Yenə, bu qradientlər tətbiq edilməzdən əvvəl, hərəkətin nə qədər yaxşı və ya pis olduğundan asılı olaraq dəyişdiriləcək.
- Nəhayət, seçilmiş hərəkəti oynayırıq və yeni müşahidəni, mükafatı, epizodun bitib-bitmədiyini, kəsilib-kəsilmədiyini və təbii ki, indicə hesabladığımız qradientləri qaytarırıq.

İndi `play_one_step()` funksiyasına əsaslanaraq çoxsaylı epizodları oynayacaq, hər bir epizod və addım üçün bütün mükafatları və qradientləri qaytaracaq başqa bir funksiya yaradaq:

```
def play_multiple_episodes(env, n_episodes, n_max_steps, model, loss_fn):
    all_rewards = []
    all_grads = []
    for episode in range(n_episodes):
        current_rewards = []
        current_grads = []
        obs, info = env.reset()
        for step in range(n_max_steps):
            obs, reward, done, truncated, grads = play_one_step(
                env, obs, model, loss_fn)
            current_rewards.append(reward)
            current_grads.append(grads)
            if done or truncated:
                break
        all_rewards.append(current_rewards)
        all_grads.append(current_grads)

    return all_rewards, all_grads
```

Bu kod mükafat siyahılarının siyahısını qaytarır: hər epizod üçün bir mükafat siyahısı, hər addım üçün bir mükafat ehtiva edir. O, həmçinin qradient siyahılarının siyahısını qaytarır: hər epizod üçün bir qradient siyahısı, hər biri hər addım üçün bir qradient tuple-i ehtiva edir və hər bir tuple hər trainable dəyişən üçün bir qradient tensoru ehtiva edir.

Alqoritm oyunu bir neçə dəfə (məsələn, 10 dəfə) oynamaq üçün `play_multiple_episodes()` funksiyasından istifadə edəcək, sonra geri qayıdıb

bütün mükafatlara baxacaq, onları diskontlaşdıracaq və normallaşdıracaq. Bunu etmək üçün daha bir neçə funksiya ehtiyacımız var; birincisi hər addımda gələcək diskontlaşdırılmış mükafatların cəmini hesablayacaq, ikincisi isə bu diskontlaşdırılmış mükafatları (yəni return-ları) çoxsaylı epizodlar üzərində ortalama çıxarmaq və standart sapmaya bölməklə normallaşdıracaq:

```
def discount_rewards(rewards, discount_factor):
    discounted = np.array(rewards)
    for step in range(len(rewards) - 2, -1, -1):
        discounted[step] += discounted[step + 1] * discount_factor
    return discounted

def discount_and_normalize_rewards(all_rewards, discount_factor):
    all_discounted_rewards = [discount_rewards(rewards, discount_factor)
                               for rewards in all_rewards]
    flat_rewards = np.concatenate(all_discounted_rewards)
    reward_mean = flat_rewards.mean()
    reward_std = flat_rewards.std()
    return [(discounted_rewards - reward_mean) / reward_std

            for discounted_rewards in all_discounted_rewards]
```

Bunun işlədiyini yoxlayaq:

```
>>> discount_rewards([10, 0, -50], discount_factor=0.8)
array([-22, -40, -50])
>>> discount_and_normalize_rewards([[10, 0, -50], [10, 20]],
...                               discount_factor=0.8)
...
array([-0.28435071, -0.86597718, -1.18910299]),
array([1.26665318, 1.0727777 ])]
```

`discount_rewards()` çağırışı tam gözlədiyimizi qaytarır (bax Şəkil 18-6). `discount_and_normalize_rewards()` funksiyasının həqiqətən də hər iki epizodda hər bir hərəkət üçün normallaşdırılmış hərəkət advantage-lərini qaytardığını yoxlaya bilərsiniz. Diqqət yetirin ki, birinci epizod ikincidən çox daha pis idi, buna görə onun normallaşdırılmış advantage-ləri hamısı mənfidir; birinci epizoddan bütün hərəkətlər pis hesab ediləcək və əksinə, ikinci epizoddan bütün hərəkətlər yaxşı hesab ediləcək.

Algoritmi işə salmağa demək olar ki, hazırıq! İndi hiperparametrləri təyin edək. 150 təlim iterasiyası işlədəcəyik, hər iterasiyada 10 epizod oynayacağıq və hər epizod ən çox 200 addım davam edəcək. 0.95 diskont amilindən istifadə edəcəyik:

```
n_iterations = 150
n_episodes_per_update = 10
n_max_steps = 200
```

```
discount_factor = 0.95
```

Bizə həmçinin bir optimizator və loss funksiyası lazımdır. 0.01 öyrənmə sürəti ilə adi bir Nadam optimizatoru kifayət edəcək və biz ikili təsnifatçı öyrətdiyimiz üçün (iki mümkün hərəkət var – sol və ya sağ) binary cross-entropy loss funksiyasından istifadə edəcəyik:

```
optimizer = tf.keras.optimizers.Nadam(learning_rate=0.01)
```

```
loss_fn = tf.keras.losses.binary_crossentropy
```

İndi təlim dövrünü qurmağa və işə salmağa hazırıq!

```
for iteration in range(n_iterations):
    all_rewards, all_grads = play_multiple_episodes(
        env, n_episodes_per_update, n_max_steps, model, loss_fn)
    all_final_rewards = discount_and_normalize_rewards(all_rewards,
                                                        discount_factor)

    all_mean_grads = []
    for var_index in range(len(model.trainable_variables)):
        mean_grads = tf.reduce_mean(
            [final_reward * all_grads[episode_index][step][var_index]
             for episode_index, final_rewards in
enumerate(all_final_rewards)
             for step, final_reward in enumerate(final_rewards)], axis=0)
        all_mean_grads.append(mean_grads)

    optimizer.apply_gradients(zip(all_mean_grads,
model.trainable_variables))
```

Bu kodu nəzərdən keçirək:

- Hər bir təlim iterasiyasında, bu dövr `play_multiple_episodes()` funksiyasını çağırır, 10 epizod oynayır və hər bir epizoddakı hər addım üçün mükafatları və qradientləri qaytarır.
- Sonra `discount_and_normalize_rewards()` funksiyasını çağıraraq hər bir hərəkətin normallaşdırılmış advantage dəyərini hesablayırıq, bu kodda `final_reward` adlanır. Bu, geriye baxdıqda hər bir hərəkətin nə qədər yaxşı və ya pis olduğunun ölçüsünü verir.

- Sonra, hər bir trainable dəyişən üçün, bütün epizodlar və addımlar üzərində həmin dəyişən üçün qradientlərin çəkili ortalamasını hesablayırıq, `final_reward` ilə çəkilmişdir.
- Nəhayət, bu ortalama qradientləri optimizatorlardan istifadə edərək tətbiq edirik: modelin trainable dəyişənləri tənzimlənəcək və ümid edirik ki, siyasət bir az daha yaxşı olacaq.

Və bitdik! Bu kod neyron şəbəkə siyasətini öyrədəcək və o, dirəyi arabanın üzərində tarazlaşdırmağı uğurla öyrənəcək. Hər epizod üzrə ortalama mükafat 200-ə çox yaxınlaşacaq. Defolt olaraq, bu environment üçün maksimum budur. Uğur!

Sadə policy gradients alqoritmi CartPole tapşırığını həll etdi, lakin daha böyük və daha mürəkkəb tapşırıqlara yaxşı miqyaslanmazdı. Həqiqətən də, o, yüksək dərəcədə *sample inefficient* (nümunə səmərəsiz) dir, yəni əhəmiyyətli irəliləyiş əldə etməzdən əvvəl oyunu çox uzun müddət kəşf etməlidir. Bu, gördüyümüz kimi, hər bir hərəkətin advantage dəyərini qiymətləndirmək üçün çoxsaylı epizodlar işlətməli olması ilə bağlıdır. Lakin bu, daha güclü alqoritmlərin, məsələn, *actor-critic* alqoritmlərinin (bu fəslin sonunda qısaca müzakirə edəcəyik) əsasını təşkil edir.

İPUCU

Tədqiqatçılar agent əvvəlcə environment haqqında heç nə bilmədikdə belə yaxşı işləyən alqoritmlər tapmağa çalışırlar. Lakin, bir məqalə yazmırsınızsa, agentə əvvəlki bilikləri daxil etməkdən çəkinməməlisiniz, çünki bu, təlimi dramatik şəkildə sürətləndirəcək. Məsələn, dirəyin mümkün qədər şaquli olması lazım olduğunu bildiyiniz üçün, dirəyin bucağına mütənəsib olaraq mənfi mükafatlar əlavə edə bilərsiniz. Bu, mükafatları daha az seyrək edəcək və təlimi sürətləndirəcək. Həmçinin, əgər artıq kifayət qədər yaxşı bir siyasətiniz varsa (məsələn, sərt kodlaşdırılmış), siyasəti yaxşılaşdırmaq üçün policy gradients istifadə etməzdən əvvəl neyron şəbəkəni onu təqlid etmək üçün öyrədə bilərsiniz.

İndi başqa bir məşhur alqoritm ailəsinə baxacağıq. PG alqoritmləri birbaşa mükafatları artırmaq üçün siyasəti optimallaşdırmağa çalışarkən, indi araşdıracağımız alqoritmlər daha az birbaşadır: agent hər bir state (vəziyyət) üçün və ya hər bir state-dəki hər bir action (hərəkət) üçün gözlənilən return-u qiymətləndirməyi öyrənir, sonra bu biliyi necə hərəkət edəcəyinə qərar vermək üçün istifadə edir. Bu alqoritmləri başa düşmək üçün əvvəlcə *Markov decision processes* (MDPs) nəzərdən keçirməliyik.

Markov Decision Processes

[Şəkil 18-7] – Sadə Markov decision process nümunəsi.

[Şəkil 18-8] – Üç vəziyyətli MDP nümunəsi.

1950-ci illərin əvvəllərində, Richard Bellman Markov decision processes-i təsvir etdi və reinforcement learning-i tədqiq etmək üçün riyazi çərçivə yaratdı. MDP-lər təbii olaraq bir agentin environment ilə qarşılıqlı əlaqəsini təsvir edir: agent vaxtaşırı müşahidələr edir, hərəkətlər edir, mükafatlar alır və environment-in vəziyyəti dəyişir. MDP-lər fərziyyə edir ki, environment-in hazırkı vəziyyəti onun bütün tarixçəsini əhatə edir; yəni gələcək yalnız hazırkı vəziyyətdən asılıdır, keçmişdən deyil. Bu, *Markov property* (Markov xassəsi) adlanır.

MDP-ni təsvir etmək üçün, vəziyyətlər, hərəkətlər, keçid ehtimalları və mükafatlar müəyyən edilməlidir. Bellman göstərdi ki, əgər agent optimal hərəkət edərsə, *Bellman optimality equation* (Bellman optimallıq tənliyi) tətbiq olunur (bax Tənlik 18-1). Bu rekursiv tənlik deyir ki, əgər agent optimal hərəkət edərsə, cari vəziyyətin optimal dəyəri, bir optimal hərəkət etdikdən sonra ortalama alacağı mükafata, üstəgəl bu hərəkətin apara biləcəyi bütün mümkün növbəti vəziyyətlərin gözlənilən optimal dəyərinə bərabərdir.

Tənlik 18-1. Bellman optimality equation

$$V(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot V(s')] \text{ for all } s$$

Bu tənlikdə:

- $T(s, a, s')$ agent a hərəkətini seçdiyini nəzərə alaraq s vəziyyətindən s' vəziyyətinə keçid ehtimalıdır. Məsələn, Şəkil 18-8-də $T(s_{-2}, a_{-1}, s_{-0}) = 0.8$.
- $R(s, a, s')$ agent a hərəkətini seçdiyini nəzərə alaraq s vəziyyətindən s' vəziyyətinə keçdikdə aldığı mükafatdır. Məsələn, Şəkil 18-8-də $R(s_{-2}, a_{-1}, s_{-0}) = +40$.
- γ diskont amilidir.

Bu tənlik birbaşa olaraq hər bir mümkün vəziyyətin optimal dəyərini dəqiq qiymətləndirə bilən bir alqoritmə gətirib çıxarır: əvvəlcə bütün vəziyyət dəyəri təxminlərini sıfıra ilkinləşdirin, sonra *value iteration* (dəyər iterasiyası) alqoritmindən istifadə edərək onları iterativ olaraq yeniləyin (bax Tənlik 18-2). Diqqətəlayiq bir nəticədir ki, kifayət qədər vaxt verildikdə, bu təxminlərin optimal siyasətə uyğun gələn optimal vəziyyət dəyərlərinə yığılmasına zəmanət verilir.

Tənlik 18-2. Value iteration alqoritm

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot V_k(s')], \forall s$$

Bu tənlikdə, $V_k(s)$ alqoritmin k ci iterasiyasında s vəziyyətinin təxmin edilən dəyəridir.

QEYD

Bu alqoritm *dynamic programming* (dinamik proqramlaşdırma) nümunəsidir, mürəkkəb bir problemi iterativ olaraq həll edilə bilən altproblemlərə bölür.

Optimal vəziyyət dəyərlərini bilmək, xüsusən bir siyasəti qiymətləndirmək üçün faydalı ola bilər, lakin bizə agent üçün optimal siyasəti vermir. Xoşbəxtlikdən, Bellman optimal *state-action values* (state-action dəyərləri) qiymətləndirmək üçün çox oxşar bir alqoritm tapdı, bunlar ümumiyyətlə *Q-values* (keyfiyyət dəyərləri) adlanır. (s, a) state-action cütünün optimal Q-dəyəri, $Q^*(s, a)$ kimi qeyd olunur, agent s vəziyyətinə çatdıqdan və a hərəkətini seçdikdən sonra, lakin bu hərəkətin nəticəsini görməzdən əvvəl, bu hərəkətdən sonra optimal hərəkət edəcəyini fərz edərək ortalama gözləyə biləcəyi diskontlaşdırılmış gələcək mükafatların cəmidir.

Gəlin onun necə işlədiyinə baxaq. Yenə, bütün Q-dəyər təxminlərini sıfıra ilkinləşdirməklə başlayırsınız, sonra *Q-value iteration* (Q-dəyər iterasiyası) alqoritmindən istifadə edərək onları yeniləyirsiniz (bax Tənlik 18-3).

Tənlik 18-3. Q-value iteration alqoritmi

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma \cdot \max_{a'} Q_k(s', a')], \forall (s, a)$$

Optimal Q-dəyərləri əldə etdikdən sonra, optimal siyasəti, $\pi(s)$ qeyd olunur, təyin etmək sadədir; agent s vəziyyətində olduqda, o, həmin vəziyyət üçün ən yüksək Q-dəyərinə malik hərəkəti seçməlidir: $\pi(s) = \operatorname{argmax}_a Q^*(s, a)$.

Gəlin bu alqoritmi Şəkil 18-8-də təsvir edilən MDP-yə tətbiq edək. Əvvəlcə, MDP-ni təyin etməliyik:

```
transition_probabilities = [ # shape=[s, a, s']
    [[0.7, 0.3, 0.0], [1.0, 0.0, 0.0], [0.8, 0.2, 0.0]],
    [[0.0, 1.0, 0.0], None, [0.0, 0.0, 1.0]],
    [None, [0.8, 0.1, 0.1], None]
]
rewards = [ # shape=[s, a, s']
    [[+10, 0, 0], [0, 0, 0], [0, 0, 0]],
    [[0, 0, 0], [0, 0, 0], [0, 0, -50]],
    [[0, 0, 0], [+40, 0, 0], [0, 0, 0]]
]
```

```
possible_actions = [[0, 1, 2], [0, 2], [1]]
```

Məsələn, s_2 -dən s_0 -a a_1 hərəkətini etdikdən sonra keçid ehtimalını bilmək üçün `transition_probabilities[2][1][0]` (0.8) baxacağıq. Eynilə, müvafiq mükafatı almaq üçün `rewards[2][1][0]` (+40) baxacağıq. Və s_2 -də mümkün hərəkətlərin siyahısını almaq üçün `possible_actions[2]` (bu halda yalnız a_1 hərəkəti mümkündür) baxacağıq. Sonra, bütün Q-dəyərlərini sıfıra ilkinləşdirməliyik (mümkün olmayan hərəkətlər üçün Q-dəyərlərini $-\infty$ təyin edirik):

```
Q_values = np.full((3, 3), -np.inf) # -np.inf for impossible actions
for state, actions in enumerate(possible_actions):
```

```
    Q_values[state, actions] = 0.0 # for all possible actions
```

İndi Q-value iteration alqoritmini işə salaq. Q, Tənlik 18-3-ü bütün Q-dəyərlərinə, hər bir vəziyyət və hər bir mümkün hərəkət üçün təkrar-təkrar tətbiq edir:

```
gamma = 0.90 # the discount factor
for iteration in range(50):
    Q_prev = Q_values.copy()
    for s in range(3):
        for a in possible_actions[s]:
            Q_values[s, a] = np.sum([
                transition_probabilities[s][a][sp]
                * (rewards[s][a][sp] + gamma * Q_prev[sp].max())
                for sp in range(3)
            ])
    ])
```

Hamısı budur! Nəticədə Q-dəyərləri belə görünür:

```
>>> Q_values
array([[18.91891892, 17.02702702, 13.62162162],
       [ 0.          , -inf, -4.87971488],
       [ -inf, 50.13365013, -inf]])
```

Məsələn, agent s_0 vəziyyətində olduqda və a_1 hərəkətini seçdikdə, gözlənilən diskontlaşdırılmış gələcək mükafatların cəmi təxminən 17.0-dir.

Hər bir vəziyyət üçün, ən yüksək Q-dəyərinə malik hərəkəti tapa bilərik:

```
>>> Q_values.argmax(axis=1) # optimal action for each state
array([0, 0, 1])
```

Bu, 0.90 diskont amilindən istifadə edərkən bu MDP üçün optimal siyasəti verir: s_0 vəziyyətində a_0 hərəkətini seç, s_1 vəziyyətində a_0 hərəkətini seç (yəni yerində qal), və s_2 vəziyyətində a_1 hərəkətini seç (yeganə mümkün hərəkət). Maraqlıdır ki, diskont amilini 0.95-ə qaldırısaq, optimal siyasət dəyişir: s_1 vəziyyətində ən yaxşı hərəkət a_2 olur (oddan keç!). Bu məntiqlidir, çünki gələcək mükafatlara nə qədər çox dəyər versəniz, indi bir az ağrıya dözməyə bir o qədər hazırsınız, gələcək xoşbəxtlik vədi üçün.

Temporal Difference Learning

Diskret hərəkətləri olan reinforcement learning problemləri çox vaxt Markov decision processes kimi modelləşdirilə bilər, lakin agent əvvəlcə keçid ehtimallarının nə olduğunu bilmir ($T(s, a, s')$ -i bilmir) və mükafatların nə olacağını da bilmir ($R(s, a, s')$ -i bilmir). Mükafatları bilmək üçün hər bir vəziyyəti və hər bir keçidi ən azı bir dəfə yaşamalısınız və keçid ehtimallarının əqlabətən təxmininə sahib olmaq üçün onları bir neçə dəfə yaşamalısınız.

Temporal difference (TD) learning alqoritmi Q-value iteration alqoritmindən çox bənzəyir, lakin agentin MDP haqqında yalnız qismən biliyə malik olduğunu nəzərə almaq üçün dəyişdirilmişdir. Ümumiyyətlə, agentin əvvəlcə yalnız mümkün vəziyyətləri və hərəkətləri bildiyini fərz edirik, daha çox deyil. Agent MDP-ni kəşf etmək üçün bir *exploration policy* (kəşfiyyat siyasəti) – məsələn, tamamilə təsadüfi siyasət – istifadə edir və irəlilədikcə, TD learning alqoritmi faktiki olaraq müşahidə olunan keçidlərə və mükafatlara əsaslanaraq vəziyyət dəyərlərinin təxminlərini yeniləyir (bax Tənlik 18-4).

Tənlik 18-4. TD learning alqoritmi

$$V_{k+1}(s) \leftarrow (1 - \alpha) V_k(s) + \alpha [r + \gamma \cdot V_k(s')]]$$

və ya ekvivalent olaraq : $V_{k+1}(s) \leftarrow V_k(s) + \alpha \cdot \delta_k(s, r, s')$

burada $\delta_k(s, r, s') = r + \gamma \cdot V_k(s') - V_k(s)$

Bu tənlikdə:

- α öyrənmə sürətidir (məsələn, 0.01).
- $r + \gamma \cdot V_k(s')$ TD target adlanır.
- $\delta_k(s, r, s')$ TD error adlanır.

Bu tənliyin birinci formasını daha qısa yazmaq üçün $a \leftarrow b$ qeydindən istifadə edə bilərik, bu $a_{k+1} \leftarrow (1 - \alpha) \cdot a_k + \alpha \cdot b_k$ deməkdir. Beləliklə, Tənlik 18-4-ün birinci sətri belə yazıla bilər: $V(s) \leftarrow a [r + \gamma \cdot V(s')]$.

İPUCU

TD learning stochastic gradient descent ilə çox oxşarlıqlara malikdir, o cümlədən hər dəfə bir nümunəni idarə etməsi. Üstəlik, SGD kimi, o, yalnız öyrənmə sürətini tədricən azaltsanız, həqiqətən yığıla bilər; əks halda, optimal Q-dəyərləri ətrafında salınmağa davam edəcək.

Hər bir s vəziyyəti üçün, bu alqoritm agentin həmin vəziyyəti tərk edərkən aldığı ani mükafatların, üstəgəl optimal hərəkət edəcəyini fərz edərək daha sonra almağı gözlədiyi mükafatların çalیشان ortalamasını izləyir.

Q-Learning

Eynilə, Q-learning alqoritmi, keçid ehtimalları və mükafatlar əvvəlcə məlum olmadıqda, Q-value iteration alqoritmının uyğunlaşdırılmış versiyasıdır (bax Tənlik 18-5). Q-learning, agentin oynamasını (məsələn, təsadüfi) izləməklə və Q-dəyərlərinin təxminlərini tədricən yaxşılaşdırmaqla işləyir. Dəqiq (və ya kifayət qədər yaxın) Q-dəyər təxminləri əldə etdikdən sonra, optimal siyasət sadəcə ən yüksək Q-dəyərinə malik hərəkəti seçməkdir (yəni greedy siyasət).

Tənlik 18-5. Q-learning alqoritmi

$$Q(s, a) \leftarrow \alpha [r + \gamma \cdot \max_{a'} Q(s', a')]]$$

Hər bir (s, a) state-action cütü üçün, bu alqoritm agentin s vəziyyətini a hərəkəti ilə tərk edərkən aldığı r mükafatının, üstəgəl almağı gözlədiyi diskontlaşdırılmış gələcək mükafatların cəminin çalیشان ortalamasını izləyir. Bu cəmi qiymətləndirmək üçün, növbəti s' vəziyyəti üçün Q-dəyər təxminlərinin maksimumunu götürürük, çünki hədəf siyasətin o andan etibarən optimal hərəkət edəcəyini fərz edirik.

Gəlin Q-learning alqoritmını tətbiq edək. Əvvəlcə, agentin environment-i kəşf etməyə məcbur edəcəyik. Bunun üçün, agentin bir hərəkət yerinə yetirə bilməsi və nəticədə yaranan vəziyyəti və mükafatı əldə etməsi üçün bir addım funksiyasına ehtiyacımız var:

```
def step(state, action):
    probas = transition_probabilities[state][action]
    next_state = np.random.choice([0, 1, 2], p=probas)
    reward = rewards[state][action][next_state]

    return next_state, reward
```

İndi agentin exploration policy (kəşfiyyat siyasəti) -ni tətbiq edək. Vəziyyət fəzası kifayət qədər kiçik olduğundan, sadə bir təsadüfi siyasət kifayət edəcək. Alqoritmi kifayət qədər uzun müddət işlətsək, agent hər bir vəziyyəti çox dəfə ziyarət edəcək və həmçinin hər bir mümkün hərəkəti çox dəfə sınayacaq:

```
def exploration_policy(state):  
  
    return np.random.choice(possible_actions[state])
```

[Şəkil 18-9] – Q-Value Iteration və Q-Learning-in müqayisəsi.

İndi Q-learning tətbiq edə bilərik. Biz indicə işlətdiyimiz kodda, yerinə yetirilən siyasət (exploration policy) tamamilə təsadüfi idi, öyrədilən siyasət isə heç vaxt istifadə edilmədi. Təlimdən sonra, optimal siyasət sistematik olaraq ən yüksək Q-dəyərinə malik hərəkəti seçməyə uyğundur. Əksinə, policy gradients alqoritmi *on-policy* alqoritmidir: o, öyrədilən siyasətdən istifadə edərək dünyanı kəşf edir. Q-learning-in yalnız bir agentin təsadüfi hərəkət etməsini izləməklə optimal siyasəti öyrənmə bilməsi təəccüblüdür. Bir qolf oynamağı, müəlliminiz gözübağlı bir meymun olduqda öyrənməyi təsəvvür edin. Daha yaxşı edə bilərikmi?

Kəşfiyyat Siyasətləri (Exploration Policies)

Əlbəttə, Q-learning yalnız exploration policy MDP-ni kifayət qədər hərtərəfli kəşf etdikdə işləyə bilər. Tamamilə təsadüfi bir siyasət nəhayət hər bir vəziyyəti və hər bir keçidi çox dəfə ziyarət edəcəyinə zəmanət versə də, bunu etmək üçün son dərəcə uzun zaman tələb oluna bilər. Buna görə, daha yaxşı bir seçim *ε-greedy policy* (ϵ -epsilondur) istifadə etməkdir: hər addımda ϵ ehtimalı ilə təsadüfi hərəkət edir, ya da $1 - \epsilon$ ehtimalı ilə greedy hərəkət edir (yəni ən yüksək Q-dəyərinə malik hərəkəti seçir). ϵ -greedy siyasətinin (tamamilə təsadüfi siyasətlə müqayisədə) üstünlüyü ondadır ki, Q-dəyər təxminləri getdikcə yaxşılaşdıqca, environment-in maraqlı hissələrini kəşf etməyə daha çox vaxt sərf edəcək, eyni zamanda MDP-nin naməlum bölgələrini ziyarət etməyə bir az vaxt ayıracaq. ϵ üçün yüksək qiymətlə (məsələn, 1.0) başlamaq və sonra tədricən azaltmaq (məsələn, 0.05-ə qədər) olduqca yaygındır.

Alternativ olaraq, kəşfiyyat üçün yalnız şansa güvənmək əvəzinə, başqa bir yanaşma, exploration policy-ni daha əvvəl çox sınaımadığı hərəkətləri sınaımağa təşviq etməkdir. Bu, Q-dəyər təxminlərinə əlavə olunan bir bonus kimi həyata keçirilə bilər, Tənlik 18-6-da göstərildiyi kimi.

Tənlik 18-6. Exploration function istifadə edərək Q-learning

$$Q(s, a) \leftarrow \alpha [r + \gamma \cdot \max_{a'} f(Q(s', a'), N(s', a'))]$$

Bu tənlikdə:

- $N(s', a')$ a' hərəkətinin s' vəziyyətində neçə dəfə seçildiyini sayır.
- $f(Q, N)$ bir *exploration function* (kəşfiyyat funksiyası)dır, məsələn, $f(Q, N) = Q + \kappa / (1 + N)$, burada κ naməlumə nə qədər cəlb olunduğunu ölçən bir maraqlı hiperparametridir.

Approximate Q-Learning və Deep Q-Learning

Q-learning-in əsas problemi, çoxlu sayda vəziyyət və hərəkəti olan böyük (və ya hətta orta) MDP-lərə yaxşı miqyaslanmamasıdır. Məsələn, *Ms. Pac-Man* oynamaq üçün bir agent öyrətmək üçün Q-learning istifadə etmək istədiyinizi fərz edin (bax Şəkil 18-1). *Ms. Pac-Man*-in yeyə biləcəyi təxminən 150 pellet var, onların hər biri mövcud və ya yox (yəni artıq yeyilib) ola bilər. Beləliklə, mümkün vəziyyətlərin sayı $2^{150} \approx 10^{45}$ -dən böyükdür. Bütün xəyalların və *Ms. Pac-Man*-in bütün mümkün mövqelərinin bütün kombinasiyalarını əlavə etsəniz, mümkün vəziyyətlərin sayı planetimizdəki atomların sayından böyük olur, buna görə hər bir fərdi Q-dəyəri üçün təxmin izləmək qətiyyəən mümkün deyil.

Həll, idarə oluna bilən sayda parametrlə (parametr vektoru θ ilə verilir) hər hansı bir (s, a) state-action cütünün Q-dəyərini təqrib edən bir $Q_\theta(s, a)$ funksiyasını tapmaqdır. Buna *approximate Q-learning* deyilir. İllər boyu, vəziyyətdən çıxarılan əl işi xüsusiyyətlərin (məsələn, ən yaxın xəyalların məsafələri, onların istiqamətləri və s.) xətti kombinasiyalarından istifadə edərək Q-dəyərlərini qiymətləndirmək tövsiyə olunurdu, lakin 2013-cü ildə DeepMind göstərdi ki, deep neural networks istifadə etmək, xüsusən mürəkkəb problemlər üçün daha yaxşı işləyə bilər və heç bir feature engineering tələb etmir. Q-dəyərlərini qiymətləndirmək üçün istifadə edilən DNN *deep Q-network* (DQN) adlanır və approximate Q-learning üçün DQN istifadə etmək *deep Q-learning* adlanır.

Bəs DQN-ni necə öyrədə bilərik? Yaxşı, DQN tərəfindən verilmiş (s, a) state-action cütü üçün hesablanan təqribi Q-dəyərini nəzərdən keçirin. Bellman sayəsində, biz bilirik ki, bu təqribi Q-dəyərinin s vəziyyətində a hərəkətini etdikdən sonra faktiki müşahidə etdiyimiz mükafat r -ə, üstəgəl o andan etibarən optimal oynamağın diskontlaşdırılmış dəyərinə mümkün qədər yaxın olmasını istəyirik. Bu gələcək diskontlaşdırılmış mükafatların cəmini qiymətləndirmək üçün, biz sadəcə DQN-ni növbəti s' vəziyyətində, bütün mümkün a' hərəkətləri üçün işlədə bilərik. Hər bir mümkün hərəkət üçün təqribi gələcək Q-dəyəri alırıq. Sonra ən yüksəyini seçirik (çünki optimal oynayacağımızı fərz edirik) və onu diskontlaşdırırıq, bu da bizə gələcək diskontlaşdırılmış mükafatların cəminin təxminini verir. Mükafat r -i və gələcək diskontlaşdırılmış dəyər təxminini toplamaqla, (s, a) state-action cütü üçün bir hədəf Q-dəyəri $y(s, a)$ alırıq, Tənlik 18-7-də göstərildiyi kimi.

Tənlik 18-7. Target Q-value

$$y(s, a) = r + \gamma \cdot \max_{a'} Q_{\theta}(s', a')$$

Bu hədəf Q-dəyəri ilə, biz istənilən gradient descent alqoritmindən istifadə edərək bir təlim addımı işlədə bilərik. Xüsusilə, biz ümumiyyətlə təxmin edilən Q-dəyəri $Q_{\theta}(s, a)$ ilə hədəf Q-dəyəri $y(s, a)$ arasındakı kvadrat səhvi və ya alqoritmin böyük səhvlərə həssaslığını azaltmaq üçün Huber loss-u minimuma endirməyə çalışırıq. Və bu, deep Q-learning alqoritmidir! Gəlin CartPole environment-ni həll etmək üçün onu necə tətbiq edəcəyimizi görək.

Deep Q-Learning-in Tətbiqi

Bizə lazım olan ilk şey bir deep Q-network-dür. Teorik olaraq, bizə giriş kimi state-action cütünü qəbul edən və təqribi Q-dəyərini çıxaran bir neural net lazımdır. Lakin praktikada, yalnız bir state-i giriş kimi qəbul edən və hər bir mümkün hərəkət üçün bir təqribi Q-dəyəri çıxaran bir neural net istifadə etmək daha səmərəlidir. CartPole environment-ni həll etmək üçün çox mürəkkəb bir neural netə ehtiyacımız yoxdur; bir neçə gizli qat kifayət edəcək:

```
input_shape = [4] # == env.observation_space.shape
n_outputs = 2 # == env.action_space.n

model = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation="elu", input_shape=input_shape),
    tf.keras.layers.Dense(32, activation="elu"),
    tf.keras.layers.Dense(n_outputs)
])
```

Bu DQN-dən istifadə edərək bir hərəkət seçmək üçün, ən böyük proqnozlaşdırılan Q-dəyərinə malik hərəkəti seçirik. Agentin environment-i kəşf etməsini təmin etmək üçün, biz ϵ -greedy siyasətindən istifadə edəcəyik (yəni ϵ ehtimalı ilə təsadüfi bir hərəkət seçəcəyik):

```
def epsilon_greedy_policy(state, epsilon=0):
    if np.random.rand() < epsilon:
        return np.random.randint(n_outputs) # random action
    else:
        Q_values = model.predict(state[np.newaxis], verbose=0)[0]

        return Q_values.argmax() # optimal action according to the DQN
```

DQN-ni yalnız ən son təcrübələrə əsaslanaraq öyrətmək əvəzinə, biz bütün təcrübələri bir *replay buffer* (və ya *replay memory*) -də saxlayacağıq və hər bir təlim iterasiyasında

ondan təsadüfi bir təlim batch-i nümunə götürəcəyik. Bu, bir təlim batch-dəki təcrübələr arasındakı korrelyasiyaları azaltmağa kömək edir, bu da təlimə böyük kömək edir. Bunun üçün, biz sadəcə bir double-ended queue (deque) istifadə edəcəyik:

```
from collections import deque
```

```
replay_buffer = deque(maxlen=2000)
```

İPUCU

Bir *deque* hər iki ucundan elementlərin səmərəli şəkildə əlavə edilə və ya çıxarıla biləcəyi bir növbədir. Növbənin ucundan elementlərin daxil edilməsi və silinməsi çox sürətlidir, lakin təsadüfi giriş növbə uzun olduqda yavaş ola bilər. Əgər çox böyük bir replay buffer-a ehtiyacınız varsa, bunun əvəzinə bir dairəvi buffer (circular buffer) istifadə etməlisiniz (tətbiq üçün notebook-a baxın) və ya DeepMind-in Reverb kitabxanasına baxın.

Hər bir təcrübə altı elementdən ibarət olacaq: bir state s , agentin etdiyi hərəkət a , nəticədə yaranan mükafat r , çatdığı növbəti state s' , epizodun həmin nöqtədə bitdiyini göstərən Boolean (done) və nəhayət, epizodun həmin nöqtədə kəsilib-kəsilmədiyini göstərən başqa bir Boolean (truncated). Replay buffer-dan təsadüfi bir təcrübə batch-i nümunə götürmək üçün kiçik bir funksiya ehtiyacımız olacaq. O, altı təcrübə elementinə uyğun gələn altı NumPy massivi qaytaracaq:

```
def sample_experiences(batch_size):
    indices = np.random.randint(len(replay_buffer), size=batch_size)
    batch = [replay_buffer[index] for index in indices]
    return [
        np.array([experience[field_index] for experience in batch])
        for field_index in range(6)
    ] # [states, actions, rewards, next_states, done, truncateds]
```

ϵ -greedy siyasətindən istifadə edərək tək bir addım oynayacaq, sonra nəticədə yaranan təcrübəni replay buffer-da saxlayacaq bir funksiya da yaradaq:

```
def play_one_step(env, state, epsilon):
    action = epsilon_greedy_policy(state, epsilon)
    next_state, reward, done, truncated, info = env.step(action)
    replay_buffer.append((state, action, reward, next_state, done,
truncated))

    return next_state, reward, done, truncated, info
```

Nəhayət, replay buffer-dan bir batch təcrübə nümunəsi götürəcək və DQN-ni bu batch üzərində tək bir gradient descent addımı yerinə yetirməklə öyrədəcək son bir funksiya yaradaq:

```
batch_size = 32
discount_factor = 0.95
optimizer = tf.keras.optimizers.Nadam(learning_rate=1e-2)
loss_fn = tf.keras.losses.mean_squared_error

def training_step(batch_size):
    experiences = sample_experiences(batch_size)
    states, actions, rewards, next_states, dones, truncateds = experiences
    next_Q_values = model.predict(next_states, verbose=0)
    max_next_Q_values = next_Q_values.max(axis=1)
    runs = 1.0 - (dones | truncateds) # episode is not done or truncated
    target_Q_values = rewards + runs * discount_factor * max_next_Q_values
    target_Q_values = target_Q_values.reshape(-1, 1)
    mask = tf.one_hot(actions, n_outputs)
    with tf.GradientTape() as tape:
        all_Q_values = model(states)
        Q_values = tf.reduce_sum(all_Q_values * mask, axis=1,
keepdims=True)
        loss = tf.reduce_mean(loss_fn(target_Q_values, Q_values))
        grads = tape.gradient(loss, model.trainable_variables)

    optimizer.apply_gradients(zip(grads, model.trainable_variables))
```

Bu kodda nələr baş verir:

- Əvvəlcə bəzi hiperparametrləri təyin edirik və optimizatoru və loss funksiyasını yaradıırıq.
- Sonra `training_step()` funksiyasını yaradıırıq. O, bir batch təcrübə nümunəsi götürməklə başlayır, sonra DQN-dən istifadə edərək hər bir təcrübənin növbəti vəziyyətində hər bir mümkün hərəkət üçün Q-dəyərini proqnozlaşdırır. Agentin optimal oynayacağını fərz etdiyimiz üçün, biz yalnız hər bir növbəti vəziyyət üçün maksimum Q-dəyərini saxlayırıq. Sonra, Tənlik 18-7-dən istifadə edərək hər bir təcrübənin state-action cütü üçün hədəf Q-dəyərini hesablayırıq.
- Biz DQN-dən istifadə edərək hər bir yaşanmış state-action cütü üçün Q-dəyərini hesablamaq istəyirik, lakin DQN digər mümkün hərəkətlər üçün də Q-dəyərlərini çıxaracaq, yalnız agent tərəfindən faktiki olaraq seçilən hərəkət üçün deyil. Beləliklə, biz lazım olmayan bütün Q-dəyərlərini maskalamalıyıq. `tf.one_hot()` funksiyası hərəkət indeksləri massivini belə bir maskaya çevirməyə imkan verir. Məsələn, əgər ilk üç təcrübə müvafiq olaraq 1, 1, 0 hərəkətlərini ehtiva edirsə, maska `[[0, 1], [0, 1], [1, 0],...]` ilə başlayacaq. Sonra

DQN-nin çıxışını bu maska ilə vura bilərik və bu, istəmədiyimiz bütün Q-dəyərlərini sıfırlayacaq. Daha sonra 1-ci ox üzərində cəmləyərək bütün sıfırlardan qurtuluruq, yalnız yaşanmış state-action cütlərinin Q-dəyərlərini saxlayırıq. Bu, bizə `Q_values` tensorunu verir, batch-dəki hər bir təcrübə üçün bir proqnozlaşdırılan Q-dəyəri ehtiva edir.

- Sonra, loss-u hesablayırıq: yaşanmış state-action cütləri üçün hədəf və proqnozlaşdırılan Q-dəyərləri arasındakı ortalama kvadrat səhvdir.

Nəhayət, modelin trainable dəyişənlərinə görə loss-u minimuma endirmək üçün gradient descent addımını yerinə yetiririk.

Bu, ən çətin hissə idi. İndi modeli öyrətmək sadədir:

```
for episode in range(600):
    obs, info = env.reset()
    for step in range(200):
        epsilon = max(1 - episode / 500, 0.01)
        obs, reward, done, truncated, info = play_one_step(env, obs,
epsilon)
        if done or truncated:
            break
    if episode > 50:
        training_step(batch_size)
```

Biz 600 epizod işlədirik, hər biri maksimum 200 addım üçün. Hər addımda, əvvəlcə ϵ -greedy siyasəti üçün epsilon dəyərini hesablayırıq: 500 epizoddan bir az az müddətdə 1-dən 0.01-ə qədər xətti olaraq azalacaq. Sonra `play_one_step()` funksiyasını çağırırıq, o, ϵ -greedy siyasətindən istifadə edərək bir hərəkət seçəcək, sonra onu yerinə yetirəcək və təcrübəni replay buffer-da qeyd edəcək. Epizod bitərsə və ya kəsilsə, dövrü tərk edirik. Nəhayət, əgər 50-ci epizoddan sonrasına, replay buffer-dan nümunə götürülmüş bir batch üzərində modeli öyrətmək üçün `training_step()` funksiyasını çağırırıq. Təlim etmədən çoxlu epizod oynamamızın səbəbi, replay buffer-ın doldurulması üçün bir az vaxt verməkdir (əgər kifayət qədər gözləməsək, replay buffer-da kifayət qədər müxtəliflik olmayacaq).

[Şəkil 18-10] – CartPole-da Deep Q-Learning-in təlim əyrisi. Ortalama mükafat 200-ə yaxınlaşır.

Bu təlim prosesi, adətən, kifayət qədər epizoddan sonra 200 mükafatına çatmağa meyllidir, lakin bəzən müvəffəq olmur (bu, hiperparametrlərə və təsadüfi toxumlara çox həssasdır).

QEYD

Reinforcement learning etibarlılığı ilə məşhurdur, əsasən təlim qeyri-sabitlikləri və hiperparametr dəyərlərinin və təsadüfi toxumların seçiminə böyük həssaslığı səbəbindən. Tədqiqatçı Andrej Karpathy-nin dediyi kimi, "[Supervised learning] işləmək istəyir. [...] RL işləməyə məcbur edilməlidir". Sizə vaxt, səbir, əzmkarlıq və bəlkə də bir az şans lazım olacaq. Bu, RL-nin adi deep learning (məsələn, convolutional nets) qədər geniş qəbul edilməməsinin əsas səbəblərindən biridir. Lakin, AlphaGo və Atari oyunlarından kənarda bir neçə real dünya tətbiqi var: məsələn, Google RL-dən datacenter xərclərini optimallaşdırmaq üçün istifadə edir və o, bəzi robotika tətbiqlərində, hiperparametr tənzimləməsi üçün və recommender systems-da istifadə olunur.

Siz niyə loss-u çəkmədiyimizi maraqlandıra bilərsiniz. Məlum olur ki, loss modelin performansının pis göstəricisidir. Loss azala bilər, lakin agent daha pis işləyə bilər (məsələn, agent environment-in kiçik bir bölgəsində ilişib qaldıqda və DQN bu bölgəni overfitting etməyə başladıqda). Əksinə, loss arta bilər, lakin agent daha yaxşı işləyə bilər (məsələn, DQN Q-dəyərlərini aşağı qiymətləndirirdisə və proqnozlarını düzgün artırmağa başlayırsa, agent yəqin ki, daha yaxşı işləyəcək, daha çox mükafat alacaq, lakin loss arta bilər, çünki DQN hədəfləri də təyin edir, onlar da daha böyük olacaq). Buna görə, mükafatları çəkmək daha yaxşıdır.

İndiyə qədər istifadə etdiyimiz əsas deep Q-learning algoritmi Atari oyunlarını oynamağı öyrənmək üçün çox qeyri-sabit olardı. Bəs DeepMind bunu necə etdi? Yaxşı, onlar algoritmi dəyişdirdilər!

Deep Q-Learning Variantları

Gəlin deep Q-learning algoritminin təlimi sabitləşdirə və sürətləndirə bilən bir neçə variantına baxaq.

Fixed Q-value Targets (Sabit Q-dəyər Hədəfləri)

Əsas deep Q-learning algoritmində, model həm proqnozlar vermək, həm də öz hədəflərini təyin etmək üçün istifadə olunur. Bu, öz quyruğunu qovan bir itə bənzər bir vəziyyətə səbəb ola bilər. Bu geribildirim dövrəsi şəbəkəni qeyri-sabit edə bilər: o, ayrılı, salına, dona bilər və s. Bu problemi həll etmək üçün, 2013-cü il məqaləsində DeepMind tədqiqatçıları bir əvəzinə iki DQN istifadə etdilər: birincisi *online model* (onlayn model) dir, hər addımda öyrənir və agentin hərəkət etdirmək üçün istifadə olunur, digəri isə *target model* (hədəf model) dir, yalnız hədəfləri təyin etmək üçün istifadə olunur. Target model sadəcə online model-in bir klonudur:


```
target = tf.keras.models.clone_model(model) # clone the model's architecture
```

```
target.set_weights(model.get_weights()) # copy the weights
```

Sonra, `training_step()` funksiyasında, növbəti vəziyyətlərin Q-dəyərlərini hesablayarkən online model əvəzinə target model-i istifadə etmək üçün yalnız bir sətiri dəyişdirməliyik:

```
next_Q_values = target.predict(next_states, verbose=0)
```

Nəhayət, təlim dövrəsində, online modelin çəkirlərini target modelə müntəzəm olaraq (məsələn, hər 50 epizoddan bir) kopyalamalıyıq:

```
if episode % 50 == 0:
```

```
    target.set_weights(model.get_weights())
```

Target model online modeldən daha az yeniləndiyi üçün, Q-dəyər hədəfləri daha sabitdir, əvvəllər müzakirə etdiyimiz geribildirim dövrəsi zəifləyir və onun təsirləri daha az şiddətli olur. Bu yanaşma, DeepMind tədqiqatçılarının 2013-cü il məqaləsindəki əsas töhfələrindən biri idi, agentlərə xam piksellərdən Atari oyunlarını oynamağı öyrənməyə imkan verdi. Təlimi sabitləşdirmək üçün onlar 0.00025 kimi kiçik bir öyrənmə sürəti istifadə etdilər, target modeli yalnız hər 10,000 addımdan bir yenilədilər (50 əvəzinə) və 1 milyon təcrübə ilə çox böyük bir replay buffer istifadə etdilər. Onlar epsilonı çox yavaş, 1 milyon addımda 1-dən 0.1-ə endirdilər və alqoritm 50 milyon addım işlətdilər. Üstəlik, onların DQN-si dərin bir convolutional net idi.

İndi DQN-nin bir daha ən müasir vəziyyəti üstələyən başqa bir variantına baxaq.

Double DQN

2015-ci ildə DeepMind tədqiqatçıları DQN alqoritmini dəyişdirdilər, performansını artırdılar və təlimi bir qədər sabitləşdirdilər. Bu variantı *double DQN* adlandırdılar. Yeniləmə, target şəbəkənin Q-dəyərlərini həddən artıq qiymətləndirməyə meyilli olduğu müşahidəsinə əsaslanırdı. Həqiqətən, tutaq ki, bütün hərəkətlər eyni dərəcədə yaxşıdır: target model tərəfindən təxmin edilən Q-dəyərləri eyni olmalıdır, lakin onlar təqribi olduğundan, bəziləri təsadüfən digərlərindən bir qədər böyük ola bilər. Target model hər zaman ən böyük Q-dəyərini seçəcək, bu da ortalama Q-dəyərindən bir qədər böyük olacaq, çox güman ki, həqiqi Q-dəyərini həddən artıq qiymətləndirəcək (bir hovuzun dərinliyini ölçərkən ən hündür təsadüfi dalğanın hündürlüyünü saymaq

kimi). Bunu düzəltmək üçün tədqiqatçılar, növbəti vəziyyətlər üçün ən yaxşı hərəkətləri seçərkən target model əvəzinə online modeldən, bu ən yaxşı hərəkətlər üçün Q-dəyərlərini qiymətləndirmək üçün isə yalnız target modeldən istifadə etməyi təklif etdilər. Budur yenilənmiş `training_step()` funksiyası:

```
def training_step(batch_size):
    experiences = sample_experiences(batch_size)
    states, actions, rewards, next_states, dones, truncateds = experiences
    next_Q_values = model.predict(next_states, verbose=0) # ≠
    target.predict()
    best_next_actions = next_Q_values.argmax(axis=1)
    next_mask = tf.one_hot(best_next_actions, n_outputs).numpy()
    max_next_Q_values = (target.predict(next_states, verbose=0) * next_mask
                        ).sum(axis=1)

    [...] # the rest is the same as earlier
```

Bir neçə ay sonra, DQN alqoritminə daha bir təkmilləşdirmə təklif edildi; növbətinə baxaq.

Prioritized Experience Replay

Replay buffer-dan təcrübələri *bərabər* nümunə götürmək əvəzinə, niyə vacib təcrübələri daha tez-tez nümunə götürməyə? Bu fikir *importance sampling* (IS) və ya *prioritized experience replay* (PER) adlanır və 2015-ci ildə DeepMind tədqiqatçıları tərəfindən (yenə!) təqdim edilmişdir.

Daha dəqiq desək, təcrübələr sürətli öyrənmə irəliləyişinə səbəb olma ehtimalı yüksək olduqda "vacib" hesab olunur. Bəs bunu necə qiymətləndirə bilərik? Bir ağlabatan yanaşma, TD error-un böyüklüyünü ölçməkdir $\delta = r + \gamma \cdot V(s') - V(s)$. Böyük bir TD error, bir keçidin (s, a, s') çox təəccüblü olduğunu və buna görə öyrənməyə dəyər olduğunu göstərir. Bir təcrübə replay buffer-da qeyd edildikdə, onun prioriteti çox böyük bir dəyərə təyin edilir ki, heç olmasa bir dəfə nümunə götürülməsini təmin etsin. Lakin, nümunə götürüldükdən sonra (və hər dəfə nümunə götürüldükdə), TD error δ hesablanır və bu təcrübənin prioriteti $p = |\delta|$ (üstəgəl hər bir təcrübənin sıfırdan fərqli nümunə götürmə ehtimalını təmin etmək üçün kiçik bir sabit) təyin edilir. p prioriteti ilə bir təcrübənin nümunə götürülmə ehtimalı P, p^ζ -ə mütənasibdir, burada ζ importance sampling-in nə qədər greedy olmasını idarə edən bir hiperparametrdir: $\zeta = 0$ olduqda, biz bərabər nümunə götürmə əldə edirik və $\zeta = 1$ olduqda, tam həcmdə importance sampling əldə edirik. Məqalədə müəlliflər $\zeta = 0.6$ istifadə etdilər, lakin optimal dəyər tapşırıqdan asılı olacaq.

Lakin bir hiylə var: nümunələr vacib təcrübələrə doğru meyli olduğundan, təlim zamanı bu meyli təcrübələri əhəmiyyətlərinə görə aşağı çəkilər verərək kompensasiya etməliyik, əks halda model sadəcə vacib təcrübələri overfitting edəcək. Aydın olmaq üçün, vacib təcrübələrin daha tez-tez nümunə götürülməsini istəyirik, lakin bu həmçinin təlim zamanı onlara daha aşağı çəki verməli olduğumuz deməkdir. Bunu etmək üçün, hər bir təcrübənin təlim çəkisini $w = (n P) - \beta$ olaraq təyin edirik, burada n replay buffer-dakı təcrübələrin sayıdır və β importance sampling meylini nə qədər kompensasiya etmək istədiyimizi idarə edən bir hiperparametrdir (0 heç demək deyil, 1 isə tamamilə deməkdir). Məqalədə müəlliflər təlimin əvvəlində $\beta = 0.4$ istifadə etdilər və təlimin sonunda xətti olaraq $\beta = 1$ -ə qədər artırdılar. Yenə, optimal dəyər tapşırıqdan asılı olacaq, lakin birini artırırsanız, adətən digərini də artırmaq istəyəcəksiniz.

İndi DQN alqoritminin sonuncu mühüm variantına baxaq.

Dueling DQN

Dueling DQN alqoritmi (DDQN, double DQN ilə qarışdırılmamalıdır, baxmayaraq ki, hər iki texnika asanlıqla birləşdirilə bilər) yenə 2015-ci ildə DeepMind tədqiqatçıları tərəfindən təqdim edilmişdir. Onun necə işlədiyini başa düşmək üçün, əvvəlcə qeyd etməliyik ki, bir (s, a) state-action cütünün Q-dəyəri $Q(s, a) = V(s) + A(s, a)$ kimi ifadə edilə bilər, burada $V(s)$ s vəziyyətinin dəyəri, $A(s, a)$ isə s vəziyyətində a hərəkətini etmənin, həmin vəziyyətdəki bütün digər mümkün hərəkətlərlə müqayisədə, *advantage* (üstünlük) dəyəridir. Üstəlik, bir vəziyyətin dəyəri, həmin vəziyyət üçün ən yaxşı hərəkətin a^* Q-dəyərinə bərabərdir (çünki optimal siyasətin ən yaxşı hərəkəti seçəcəyini fərz edirik), beləliklə $V(s) = Q(s, a^*)$, bu da $A(s, a^*) = 0$ deməkdir. Dueling DQN-də, model həm vəziyyətin dəyərini, həm də hər bir mümkün hərəkətin *advantage* dəyərini qiymətləndirir. Ən yaxşı hərəkətin *advantage* dəyəri 0 olmalı olduğundan, model bütün proqnozlaşdırılan *advantage* dəyərlərindən maksimum proqnozlaşdırılan *advantage* dəyərini çıxarır. Budur functional API istifadə edərək tətbiq edilmiş sadə bir DDQN modeli:

```
input_states = tf.keras.layers.Input(shape=[4])
hidden1 = tf.keras.layers.Dense(32, activation="elu")(input_states)
hidden2 = tf.keras.layers.Dense(32, activation="elu")(hidden1)
state_values = tf.keras.layers.Dense(1)(hidden2)
raw_advantages = tf.keras.layers.Dense(n_outputs)(hidden2)
advantages = raw_advantages - tf.reduce_max(raw_advantages, axis=1,
keepdims=True)
Q_values = state_values + advantages
```

```
model = tf.keras.Model(inputs=[input_states], outputs=[Q_values])
```

Alqoritmin qalan hissəsi əvvəlki kimi eynidir. Əslində, siz bir double dueling DQN qurub onu prioritized experience replay ilə birləşdirə bilərsiniz! Daha ümumiləşdirilmiş olaraq, bir çox RL texnikası birləşdirilə bilər, DeepMind 2017-ci il məqaləsində nümayiş etdirdiyi kimi: məqalənin müəllifləri altı fərqli texnikanı *Rainbow* adlanan bir agentdə birləşdirdilər, bu da ən müasir vəziyyəti böyük ölçüdə üstələdi.

Gördüyünüz kimi, deep reinforcement learning sürətlə böyüyən bir sahədir və kəşf ediləcək daha çox şey var!

Bəzi Populyar RL Alqoritmlərinin Xülasəsi

Bu fəslə bağlamazdan əvvəl, gəlin bir neçə digər populyar alqoritmə qısaca nəzər salaq:

AlphaGo

AlphaGo, Go oyununda insan çempionlarını məğlub etmək üçün dərin neyron şəbəkələrə əsaslanan *Monte Carlo tree search* (MCTS) variantından istifadə edir. MCTS 1949-cu ildə Nicholas Metropolis və Stanislaw Ulam tərəfindən icad edilmişdir. O, bir çox simulyasiya işlətməklə, cari mövqedən başlayaraq axtarış ağacını təkrar-təkrar kəşf edərək və ən perspektivli budaqlara daha çox vaxt sərf edərək ən yaxşı gedişi seçir. Əvvəllər ziyarət etmədiyi bir qovşağa çatdıqda, oyun bitənə qədər təsadüfi oynayır və hər bir ziyarət edilmiş qovşaq üçün təxminlərini (təsadüfi gedişləri istisna etməklə) yeniləyir, son nəticədən asılı olaraq hər bir təxmini artırır və ya azaldır. AlphaGo eyni prinsipə əsaslanır, lakin təsadüfi oynamaq əvəzinə gedişləri seçmək üçün bir policy network istifadə edir. Bu policy net policy gradients istifadə edərək öyrədilir. Orijinal alqoritm daha üç neyron şəbəkəni əhatə edirdi və daha mürəkkəb idi, lakin AlphaGo Zero məqaləsində sadələşdirildi, burada həm gedişləri seçmək, həm də oyun vəziyyətlərini qiymətləndirmək üçün tək bir neyron şəbəkədən istifadə edilir. AlphaZero məqaləsi bu alqoritmi ümumiləşdirərək, onu yalnız Go deyil, həm də şahmat və şogi (yapon şahmatı) oynamağa qadir etdi. Nəhayət, MuZero məqaləsi bu alqoritmi daha da təkmilləşdirərək, agent hətta oyunun qaydalarını bilmədən başlasa belə, əvvəlki iterasiyaları üstələdi!

Actor-critic alqoritmləri

Actor-critic, policy gradients-i deep Q-networks ilə birləşdirən bir RL alqoritmləri ailəsidir. Bir actor-critic agent iki neyron şəbəkə ehtiva edir: bir policy net və bir DQN. DQN normal olaraq, agentin təcrübələrindən öyrənməklə öyrədilir. Policy net isə adi

PG-dən fərqli (və daha sürətli) öyrənir: hər bir hərəkətin dəyərini çoxsaylı epizodlardan keçərək, sonra hər bir hərəkət üçün gələcək diskontlaşdırılmış mükafatları cəmləyərək və nəhayət onları normallaşdırmaq əvəzinə, agent (aktor) DQN (kritik) tərəfindən qiymətləndirilən hərəkət dəyərlərinə güvənir. Bu, bir idmançının (agent) bir məşqçinin (DQN) köməyi ilə öyrənməsinə bənzəyir.

Asynchronous advantage actor-critic (A3C)

Bu, 2016-cı ildə DeepMind tədqiqatçıları tərəfindən təqdim edilən mühüm bir actor-critic variantıdır, burada çoxsaylı agentlər paralel olaraq öyrənir, environment-in müxtəlif surətlərini kəşf edirlər. Müntəzəm olaraq, lakin asinxron (buna görə ad), hər bir agent master şəbəkəyə bəzi çəki yeniləmələrini itələyir, sonra həmin şəbəkədən ən son çəkilişi alır. Beləliklə, hər bir agent master şəbəkənin təkmilləşməsinə töhfə verir və digər agentlərin öyrəndiklərindən faydalanır. Üstəlik, Q-dəyərlərini qiymətləndirmək əvəzinə, DQN hər bir hərəkətin advantage dəyərini qiymətləndirir (buna görə addakı ikinci A), bu da təlimi sabitləşdirir.

Advantage actor-critic (A2C)

A2C, A3C alqoritminin asinxronluğu aradan qaldıran bir variantıdır. Bütün model yeniləmələri sinxronudur, buna görə qradient yeniləmələri daha böyük batch-lər üzərində yerinə yetirilir, bu da modelin GPU-nun gücündən daha yaxşı istifadə etməsinə imkan verir.

Soft actor-critic (SAC)

SAC, 2018-ci ildə Tuomas Haarnoja və digər UC Berkeley tədqiqatçıları tərəfindən təklif edilən bir actor-critic variantıdır. O, təkcə mükafatları deyil, həm də hərəkətlərinin entropiyasını maksimuma çıxarmağı öyrənir. Başqa sözlə, o, mümkün qədər çox mükafat alarkən eyni zamanda mümkün qədər gözlənilməz olmağa çalışır. Bu, agentin environment-i kəşf etməyə təşviq edir, bu da təlimi sürətləndirir və DQN qeyri-kamil təxminlər istehsal etdikdə eyni hərəkəti təkrar-təkrar yerinə yetirmə ehtimalını azaldır. Bu alqoritm heyrətamiz bir sample səmərəliliyi nümayiş etdirmişdir (əvvəlki bütün alqoritmlərdən fərqli olaraq, onlar çox yavaş öyrənirlər).

Proximal policy optimization (PPO)

John Schulman və digər OpenAI tədqiqatçıları tərəfindən bu alqoritm A2C-yə əsaslanır, lakin həddindən artıq böyük çəki yeniləmələrinin (tez-tez təlim qeyri-sabitliklərinə səbəb olan) qarşısını almaq üçün loss funksiyasını kəsir. PPO, əvvəlki *trust region policy optimization* (TRPO) alqoritminin sadələşdirilməsidir, həmçinin OpenAI tərəfindən. OpenAI, PPO alqoritminə əsaslanan OpenAI Five adlı AI-si ilə 2019-cu ilin aprelində xəbərlərə çıxdı, bu, multiplayer oyun *Dota 2* -də dünya çempionlarını məğlub etdi.

Curiosity-based exploration

RL-də təkrarlanan bir problem mükafatların seyrəkliyidir, bu da öyrənməni çox yavaş və səmərəsiz edir. Deepak Pathak və digər UC Berkeley tədqiqatçıları bu problemin öhdəsindən gəlmək üçün həyəcanverici bir yol təklif etdilər: niyə mükafatları nəzər almayaq və sadəcə agenti environment-i kəşf etmək üçün son dərəcə maraqlı etməyə? Mükafatlar beləliklə agent üçün daxili olur, environment-dən gəlmir. Eynilə, bir uşaqda maraq oyatmaq, uşağı yaxşı qiymətlər aldığı üçün mükafatlandırmaqdan daha çox yaxşı nəticələr vermə ehtimalına malikdir. Bu necə işləyir? Agent davamlı olaraq hərəkətlərinin nəticəsini proqnozlaşdırmağa çalışır və nəticənin proqnozlarına uyğun gəlmədiyi vəziyyətləri axtarır. Başqa sözlə, o, təəccüblənmək istəyir. Əgər nəticə proqnozlaşdırıla biləndirsə (darıxdırıcı), başqa yerə gedir. Lakin, əgər nəticə proqnozlaşdırıla bilməz, lakin agent onu idarə edə bilmədiyini görürsə, bir müddətdən sonra o da darıxır. Yalnız maraqla, müəlliflər bir agenti bir çox video oyunlarında öyrətməyi bacardılar: agent uduzduğu üçün heç bir cəza almasa da, oyun yenidən başlayır, bu darıxdırıcıdır, buna görə ondan qaçmağı öyrənir.

Open-ended learning (OEL)

OEL-in məqsədi sonsuz olaraq yeni və maraqlı tapşırıqları (adətən prosedur olaraq yaradılan) öyrənə bilən agentlər hazırlamaqdır. Hələ orada deyilik, lakin son bir neçə ildə heyratamiz irəliləyiş olmuşdur. Məsələn, 2019-cu ildə Uber AI-dən bir qrup tədqiqatçı tərəfindən *POET alqoritmi* təqdim edildi, bu, qabar və çuxurları olan çoxsaylı simulyasiya edilmiş 2D environment-lər yaradır və hər environment üçün bir agent öyrədir: agentin məqsədi maneələrdən qaçaraq mümkün qədər sürətli yeriməkdir. Alqoritm sadə environment-lərlə başlayır, lakin zamanla onlar tədricən çətinləşir: buna *curriculum learning* deyilir. Üstəlik, hər bir agent yalnız bir environment daxilində öyrədilsə də, o, müntəzəm olaraq bütün environment-lər üzrə digər agentlərlə rəqabət aparmalıdır. Hər bir environment-də qalib köçürülür və əvvəlki agenti əvəz edir. Bu yolla, bilik müntəzəm olaraq environment-lər arasında köçürülür və ən uyğunlaşa bilən agentlər seçilir. Nəticədə, agentlər tək bir tapşırıq üzərində öyrədilən agentlərdən daha yaxşı yeriyənlər olurlar və daha çətin environment-ləri həll edə bilirlər. Əlbəttə, bu prinsip digər environment-lərə və tapşırıqlara da tətbiq edilə bilər. Əgər OEL ilə maraqlanırsınızsa, Enhanced POET məqaləsini, həmçinin DeepMind-in bu mövzuda 2021-ci il məqaləsini mütləq yoxlayın.

İPUCU

Reinforcement learning haqqında daha çox öyrənmək istəyirsinizsə, Phil Winder-in *Reinforcement Learning* kitabını (O'Reilly) yoxlayın.

Bu fəsildə bir çox mövzunu əhatə etdik: policy gradients, Markov chains, Markov decision processes, Q-learning, approximate Q-learning və deep Q-learning və onun əsas variantları (fixed Q-value targets, double DQN, dueling DQN və prioritized experience replay) və nəhayət, bir neçə digər məşhur alqoritmə qısa nəzər saldıq. Reinforcement learning nəhəng və həyəcanverici bir sahədir, hər gün yeni ideyalar və alqoritmlər ortaya çıxır, buna görə ümid edirəm ki, bu fəsil sizin marağınızı oyatdı: kəşf ediləcək bütöv bir dünya var!

Məşqlər

1. Reinforcement learning-i necə tərif edərdiniz? Onun adi supervised və ya unsupervised learning-dən fərqi nədir?
2. Bu fəsildə qeyd olunmayan RL-nin üç mümkün tətbiqini düşünə bilərsinizmi? Onların hər biri üçün environment nədir? Agent nədir? Bəzi mümkün hərəkətlər hansılardır? Mükafatlar hansılardır?
3. Discount factor (diskont amili) nədir? Diskont amilini dəyişdirsəniz, optimal siyasət dəyişə bilərmi?
4. Bir reinforcement learning agentinin performansını necə ölçürsünüz?
5. Credit assignment problem (kredit bölgüsü problemi) nədir? O, nə vaxt baş verir? Onu necə yüngülləşdirə bilərsiniz?
6. Replay buffer istifadəsinin məqsədi nədir?
7. Off-policy RL alqoritmi nədir?
8. Policy gradients istifadə edərək OpenAI Gym-in LunarLander-v2 environment-ni həll edin.
9. Məşhur Atari *Breakout* oyununda ("ALE/Breakout-v5") insan səviyyəsindən yuxarı səviyyəyə çata bilən bir agent öyrətmək üçün double dueling DQN istifadə edin. Müşahidələr şəkillərdir. Tapşırığı sadələşdirmək üçün, onları bozqanadlıya çevirməli (yəni kanallar oxu üzərində ortalama almalı), sonra kəsməli və aşağı ölçüləndirməlisiniz, beləliklə oynamaq üçün kifayət qədər böyük, lakin daha çox olmamalıdır. Tək bir şəkil topun və çubuqların hansı istiqamətdə getdiyini söyləmir, buna görə hər bir state-i yaratmaq üçün iki və ya üç ardıcıl şəkli birləşdirməlisiniz. Nəhayət, DQN əsasən convolutional qatlardan ibarət olmalıdır.

10. Təxminən 100 dollarınız varsa, bir Raspberry Pi 3 plus bəzi ucuz robotika komponentləri ala, Pi-yə TensorFlow quraşdır və sərbəstləşə bilərsiniz! Bir nümunə üçün Lukas Biewald-ın əyləncəli yazısına baxın, ya da GoPiGo və ya BrickPi-yə nəzər salın. Sadə məqsədlərlə başlayın, məsələn, robotun ən parlaq bucağı (əgər işıq sensoru varsa) və ya ən yaxın obyekt (əgər sonar sensoru varsa) tapmaq üçün dönməsi və həmin istiqamətdə hərəkət etməsi. Sonra deep learning istifadə etməyə başlaya bilərsiniz: məsələn, əgər robotun kamerası varsa, insanları aşkar edib onlara doğru hərəkət edən bir object detection alqoritmi tətbiq etməyə cəhd edə bilərsiniz. Siz həmçinin RL-dən istifadə edərək agentin öz başına bu məqsədə çatmaq üçün mühərrikləri necə istifadə edəcəyini öyrənməsinə cəhd edə bilərsiniz. Əylənin!

Bu məşqlərin həlləri bu fəslin notebook-unun sonunda, <https://homl.info/colab3> ünvanında mövcuddur.